Below is a hackathon-grade specification + detailed design doc for a Daily Multi-Agent Intelligence System that (1) crawls configurable sources, (2) summarizes and ranks updates, (3) compiles a polished PDF digest, and (4) emails it to you + a configurable distribution list.

---

1) Product overview

Goal

Create a daily "Frontier AI Radar" that automatically tracks:

- Competitor releases (product/platform updates)
- Foundation model provider releases (model launches, API updates, pricing, eval claims)
- Latest research publications (LLMs / multimodal / agents / eval / alignment)
- Hugging Face benchmarking results (leaderboards, new SOTA claims, dataset/task-specific trends)
- ...and produces an executive + technical digest as PDF, delivered by email.

Non-goals (for hackathon scope)

- Training models
- Full web-scale crawling
- Real-time monitoring (daily batch is enough)

---

2) System requirements

Functional requirements

FR1 – Configurable sources per agent

- Each agent reads a config file (YAML/JSON) listing:
- URLs (and optional RSS feeds / API endpoints)
- Crawl frequency (default: daily)
- Crawl depth / include-exclude rules
- Parsing hints (CSS selectors, sitemap paths, “release notes” keywords)

FR2 – Robust extraction

- Must handle:
- HTML pages
- RSS/Atom
- PDF release notes (optional)
- “Changelog” pages with pagination

FR3 – Summarization

- Each agent produces:
- Top updates (bullet list)
- What changed (before/after if detectable)
- Why it matters (impact summary)
- Confidence score + citations (source URLs)

FR4 – Dedup + clustering

- Merge duplicates across sources (same announcement repeated across blog, docs, press).
- Cluster by topic: Models, APIs, Pricing, Benchmarks, Safety, Tooling.

FR5 – PDF generation

- Daily PDF includes:
- Cover + date + audience
- Executive summary (1 page)
- Deep dives by section (Competitors / Model Providers / Research / HF Benchmarks)

Appendix: source links & raw snippets (optional)

FR6 – Email delivery

Send to:

“Me” (you) + a configurable list (“select research people”)

Email includes:

- Short executive snippet inline
- PDF attachment
- Links to web dashboard

FR7 – Web UI

Minimal UI for hackathon scoring:

- Manage sources (add/edit URLs)
- Trigger run (manual) + see run history
- Browse daily digests + search
- Per-agent logs & failures

Non-functional requirements

- NFR1 Reliability: partial failure tolerant (one agent can fail without blocking digest)
- NFR2 Observability: logs, metrics, trace per URL
- NFR3 Rate limiting & robots: respect robots.txt where required; throttle per domain
- NFR4 Security: secrets vault for email creds, API keys
- NFR5 Cost control: caching + incremental diffs; avoid re-summarizing identical pages

---

3) High-level architecture

Components

Orchestrator (Scheduler + Run Manager)

Agent Workers (4 crawlers + 1 compiler/sender)

Fetcher Layer (HTTP + headless browser when needed)

Extractor Layer (HTML→text, PDF→text, metadata)

Change Detector (diff, fingerprinting, canonicalization)

Summarizer (LLM summarization + structured output)

Knowledge Store

Raw snapshots

Extracted text

Structured “Findings”

Digest Compiler

Ranking + narrative synthesis

PDF renderer

Notification Service

Email sender + retry queue

Web UI

Admin config + runs + results

Data flow (daily run)

Scheduler → Run Manager → (Agents 1–4 in parallel)

Agents produce structured findings → Dedup/Cluster → Digest Agent → PDF → Email

4) Agent specifications (detailed)

Shared agent interface

Each agent implements:

Input

run_id

agent_config (URLs, rules, rate limits)

since_timestamp (for incremental scanning)

Output

findings[] with schema:

title

date_detected

source_url

publisher

category (release / research / benchmark)

summary_short (≤ 60 words)

summary_long (bullets + detail)

why_it_matters

evidence (quoted snippets ≤ short, plus extracted metadata)

confidence (0–1)

tags[]

entities[] (company/model/dataset)

diff_hash (for dedup)

---

5) Agent #1 — Competitor Release Watcher

Purpose

Track configurable competitor sites (product blogs, changelogs, docs release notes) and summarize latest releases.

Inputs (config example)

Competitors:

name

release_urls[] (blog/changelog/docs)

rss_feeds[] (optional)

selectors (optional CSS selectors for main content)

keywords[] (e.g., “release”, “changelog”, “GA”, “beta”)

domain_rate_limit

Logic

Discover new pages (RSS preferred → else sitemap → else crawl list)

Fetch page content (HTTP; fallback to headless if JS-rendered)

Extract canonical text + metadata (title, date, author)

Detect change via fingerprint/diff against last snapshot

Summarize into structured finding (include “what changed”)

Rank by impact:

mentions of GA/pricing/API/latency/security/compliance

Emit findings

Edge cases

“Last updated” pages with no clear publication date → use diff date + heuristics

Multiple versions (v1/v2) → normalize

---

6) Agent #2 — Foundation Model Provider Release Watcher

Purpose

Track model releases, API updates, pricing changes, and evaluation claims from foundation model providers.

Sources

Configurable URLs (docs, release notes, blog, status pages). Prefer official sources.

Output focus

Model name/version

Modalities

Context length

Tool use / agents features

Pricing / quotas

Safety policy changes

Benchmarks claimed (with citations)

Extra: Claim verification hooks (optional)

If a release claims SOTA, create a “verification task” for Agent #4 (HF) to validate.

---

7) Agent #3 — Research Publication Scout

Purpose

Track the latest publications on LLMs/foundation models and summarize what matters.

Acquisition options (configurable)

arXiv categories/queries (CS.CL, CS.LG, stat.ML)

Semantic Scholar queries

OpenReview (top conferences / workshop tracks)

Curated lab blogs (as URLs)

Output focus

Core contribution (1–2 sentences)

What’s new vs prior work

Practical implications (eval, data, training, inference, agents)

Relevance score (to your org’s priorities: data, eval, safety, localization, multimodal)

Relevance scoring (hackathon win condition)

Score papers higher if they include:

New benchmarks/eval methodology

Data-centric techniques (curation, synthetic, RL, preference learning)

Agentic workflows, tool use, memory

Multimodal reasoning, video, robotics/physical AI

Safety/alignment, red-teaming, policy compliance

---

8) Agent #4 — Hugging Face Benchmark & Leaderboard Tracker

Purpose

Monitor HF for latest benchmarking results and trends across providers.

What it tracks (configurable)

Hugging Face leaderboards (general + task-specific)

Newly trending models (daily/weekly)

Evaluation datasets of interest (e.g., instruction-following, reasoning, coding, safety)

“New SOTA” claims

Output focus

Who moved up/down

Which tasks improved

Model family trends (e.g., small models catching up, multimodal jumps)

Reproducibility notes (if metadata available)

Caveats (leaderboard bias, different eval settings)

Design note

HF data often comes from APIs/structured pages; prefer official endpoints where possible, fallback to HTML.

---

9) Final Agent — Digest Compiler + PDF + Email Sender

Purpose

Combine all agent findings into one cohesive daily report.

Steps

Deduplicate findings (same URL, same diff_hash, semantic similarity)

Cluster by topic + entity (provider/model/dataset)

Rank by impact & relevance (weighted scoring)

Generate narrative

Exec summary (top 7 items)

“What changed since yesterday”

“Why it matters to us”

Render PDF

Brandable header/footer

Tables for quick scan

Clickable citations

Send email

Inline bullets

PDF attached

Link to dashboard run

---

10) Configuration design

Config file structure (recommended)

global:

run_time (e.g., 06:30 PT daily)

timezone

max_pages_per_domain

default_rate_limit

email_recipients[]

agents:

competitors: list of competitor definitions

model_providers: list

research: queries + URLs

hf_benchmarks: leaderboards + tasks

Dynamic config in UI

UI edits write back to config store (DB) and version it:

config_version

audit log (who changed what)

---

11) Orchestration & scheduling

Daily schedule

Use a scheduler (Cron / Airflow / Dagster / Prefect / Temporal)

Run agents in parallel with dependencies:

Agents 1–4 → Digest Agent

Retry strategy

Per-URL retries with exponential backoff

Domain-level circuit breaker (avoid hammering failing sites)

---

12) Storage & data model

Minimal tables/collections

sources (URL, agent, rules, rate limit)

snapshots (url, fetched_at, content_hash, raw_html/blob ref)

extractions (url, text, title, published_date, metadata)

findings (structured summaries)

runs (run_id, status, timings)

digests (pdf_path, html_path, summary, recipients)

Caching / incremental diffs

If content_hash unchanged → skip re-summarization

Keep last N snapshots per URL for diff analysis

---

13) Summarization contract (structured output)

Standard prompts (conceptual)

“Summarize this release note. Extract: What changed, why it matters, who it affects, any numbers/claims, and direct source citation.”

Guardrails

Always include citations (URLs + small snippet evidence)

Don’t invent benchmark scores—if unknown, mark “Not stated”

Confidence scoring:

High if official source + clear date + clear change

Medium if unclear date or ambiguous claims

Low if scraped repost / third-party summary

---

14) Ranking & scoring (how you win the hackathon)

Impact score formula (example)

Impact = 0.35 * Relevance + 0.25 * Novelty + 0.20 * Credibility + 0.20 * Actionability

Relevance: matches your tracked entities/topics

Novelty: new vs last 7 days

Credibility: official sources > reposts

Actionability: includes concrete changes (API/pricing/eval) and implications

---

15) Web UI (hackathon-ready)

Pages

Dashboard

Last run status

Top 10 updates today

Download PDF

Sources

Add/edit URLs

Agent assignment

Rate limits / include-exclude rules

Runs

Timeline

Errors per agent

Findings Explorer

Filter by entity/provider/topic

Compare with yesterday (diff view)

Digest Archive

Past PDFs

Search digests

Bonus UI features (prize material)

“What changed” diff viewer for key pages

“SOTA Watch” leaderboard movement chart

“Entity Heatmap” (providers vs topics)

16) Security, compliance, and responsible crawling

Respect robots.txt and ToS where applicable

Rate-limit aggressively; identify user agent

Store secrets in vault (email creds/API keys)

Avoid scraping behind auth unless permitted and configured

PII: do not collect; if detected, redact

---

17) Testing & evaluation plan

Unit tests

HTML extraction per template

Date parsing

Dedup hashing

PDF rendering

Integration tests

Run end-to-end against a “test corpus” of static pages

Failure injection: 404, timeouts, JS-only pages

Quality tests

Hallucination checks:

Any numeric claim must map to evidence snippet

Consistency:

Same article summarized similarly across runs if unchanged

---

18) Hackathon execution plan (2 days)

Day 1: Core pipeline

Config store + scheduler + Agent 1 basic crawl + structured findings

Day 2: Add Agents 2–4 + dedup

Implement change detection + storage

Day 3: PDF + Email + UI polish

Dashboard + Sources CRUD + digest archive

Demo script (what judges want)

Add a new competitor URL in UI → run now → see findings → download PDF → email received

---

19) Deliverables checklist

- ✅ Multi-agent pipeline (1–4 + digest agent)
- ✅ Configurable URLs per agent
- ✅ Daily scheduler + manual run
- ✅ Dedup + ranking
- ✅ PDF digest (branded template-ready)
- ✅ Email distribution
- ✅ UI with sources + runs + archive
- ✅ Observability + retries

---

20) Suggested tech stack (practical + fast)

Orchestrator: Prefect / Dagster / Temporal (hackathon: Prefect is quick)

Crawler: httpx + readability + Playwright fallback

Parsing: BeautifulSoup + trafilatura (HTML→text)

Storage: Postgres + S3-compatible object store (or SQLite for hackathon)

Embeddings for dedup: local embedding model or API

PDF: ReportLab or WeasyPrint (HTML→PDF)

Email: SMTP or SendGrid

UI: Next.js + FastAPI, or Streamlit for fastest demo